

Configure request routing

 View Markdown

Learn how to deploy multiple LLMs from a single endpoint and configure request routing strategies to optimize cache locality and improve LLM serving performance.

Understand request routing

Request routing in Ray Serve LLM operates at two levels:

- **Application-level routing:** When you deploy multiple models, Ray Serve LLM directs requests to the appropriate deployment based on the model ID specified in the request. Each model runs as a separate deployment with its own replicas.
- **Deployment-level routing:** Within each model deployment, Ray Serve LLM selects which replica handles each request. This is where strategies like prefix-aware routing and custom routing policies apply.

Ray Serve LLM supports multiple deployment-level routing strategies. The default router distributes load evenly across replicas, but specialized routers can optimize for cache locality when workloads have shared prefixes.

NOTE

By default, Ray Serve LLM uses the [power-of-two routing strategy](#), which selects two replicas at random and routes the request to the one with fewer pending requests.

For an overview of routing architecture and available strategies, see the [Ray Serve LLM routing documentation](#).

Deploy multiple LLMs

 Ask AI

Application-level routing enables you to deploy multiple LLMs from a single endpoint, allowing clients to choose which model to use by specifying the model ID in their requests. The router

directs each request to the appropriate deployment based on the `model` parameter. This approach simplifies deployment management and enables use cases such as A/B testing, model comparison, or offering different models for different tasks.

Configure and deploy multiple LLM deployments

To deploy multiple LLMs, create multiple `LLMConfig` objects and pass them to `build_openai_app`. Each model can have independent configuration for accelerator type, autoscaling, and engine parameters.

```
# multi_llm_app.py
from ray.serve.llm import LLMConfig, build_openai_app

llama_config = LLMConfig(
    model_loading_config=dict(
        model_id="my-llama",
        model_source="meta-llama/Llama-3.1-8B-Instruct",
    ),
    ...
)

mistral_config = LLMConfig(
    model_loading_config=dict(
        model_id="my-mistral",
        model_source="mistralai/Mistral-7B-Instruct-v0.3",
    ),
    ...
)

app = build_openai_app({"llm_configs": [llama_config, mistral_config]})
```

The `model_id` field uniquely identifies each model and determines how clients select the model in their requests.

Launch your application with Ray Serve LLM or as an Anyscale service:

[Ray Serve LLM](#)

[Anyscale service](#)

 Ask AI

Deploy your prefix-aware routing application locally with Ray Serve LLM:

```
serve run multi_llm_app:app
```

Query multiple LLMs

Once deployed, clients specify which model to use by setting the `model` parameter in their request. The router automatically directs the request to the appropriate model deployment.

```
from openai import OpenAI

client = OpenAI(
    base_url="https://your-service-url/v1",
    api_key="your-api-key"
)

# Query the Llama model
response = client.chat.completions.create(
    model="my-llama",
    messages=[{"role": "user", "content": "Hello Llama!"}]
)

# Query the Mistral model
response = client.chat.completions.create(
    model="my-mistral",
    messages=[{"role": "user", "content": "Hello Mistral!"}]
)
```

Each model deployment operates independently with its own replicas, autoscaling configuration, and resource allocation.

Configure prefix-aware routing

Prefix-aware routing is a replica routing strategy that optimizes cache locality by routing requests with similar prefixes to the same replica. Because each replica has its own KV cache, routing requests with shared prefixes to the same replica maximizes cache reuse and can improve throughput for workloads with shared prefixes such as system prompts in chatbots or



Ask AI

few-shot examples in classification tasks. For more details, see the [Ray Serve LLM prefix-aware routing guide](#).

Configure and deploy prefix-aware routing

To configure prefix-aware routing with Ray Serve LLM, add a `request_router_config` section to your deployment configuration and set the `request_router_class` to `PrefixCacheAffinityRouter`. You can configure router parameters in `request_router_kwargs`.

```
# my_prefix_aware_app.py
from ray import serve
from ray.serve.llm import LLMConfig, build_openai_app
from ray.serve.llm.request_router import PrefixCacheAffinityRouter

llm_config = LLMConfig(
    ...
    deployment_config=dict(
        ...
        request_router_config=dict(
            request_router_class=PrefixCacheAffinityRouter,
            request_router_kwargs={
                "match_rate_threshold": 0.1, # Require 10% match rate for
prefix routing
                "imbalanced_threshold": 10,
            },
        ),
    ),
)

app = build_openai_app({"llm_configs": [llm_config]})
```

For more details on configuring your prefix-aware router, see the [Ray Serve LLM documentation](#).

Launch your application with Ray Serve LLM or as an Anyscale service:



[Ray Serve LLM](#) Anyscale service

Deploy your prefix-aware routing application locally with Ray Serve LLM:

```
serve run my_prefix_aware_app:app
```

Once deployed, query your service normally using the OpenAI-compatible API. The prefix-aware router automatically handles request distribution to optimize cache locality. The router directs requests with shared prefixes, such as identical system prompts, to the same replica when possible, allowing the engine to reuse cached KV entries.

Apply best practices

To make the most of prefix-aware routing in your production deployments, consider applying the following best practices and tuning strategies to maximize cache locality and system performance:

Balance load and cache locality

The `imbalanced_threshold` parameter controls when the router prioritizes load balancing over cache locality. Lower values favor load distribution, higher values favor cache hits. Tune this based on your latency requirements and load patterns.

Configure prefix match threshold

The `match_rate_threshold` parameter sets the minimum prefix match rate required to use prefix cache-aware routing, with valid values from 0.0 to 1.0. Higher values require stronger prefix matches before routing for cache locality. Increase this value to ensure routing decisions favor only strong prefix matches, or decrease it to enable cache-aware routing for weaker prefix similarities.

Consider workload characteristics

Prefix-aware routing provides the most benefit when many requests share long prefixes, such as system prompts or shared context documents. For workloads with short or highly diverse prefixes, the [default power-of-two router](#) may perform better.

 Ask AI

Configure memory management

Enable automatic eviction of old prefix entries with `do_eviction=True` to manage memory usage in high-traffic deployments. The router approximates the LLM engine's eviction policy, keeping its prefix cache synchronized with the engine's actual KV cache to prevent routing based on stale prefix information.

Configure custom routing policy

For advanced use cases requiring custom replica routing strategies beyond prefix-aware routing, you can implement your own router by extending Ray Serve's [RequestRouter](#) interface. Custom routers control which replica of a deployment handles each request.

For implementation guidance, see the [Ray Serve LLM routing documentation](#).

Select the right routing policy

Choose your replica routing strategy based on your workload characteristics and performance requirements. These policies control which replica handles each request within a model deployment.

Use default routing when

The [default power-of-two router](#) works well for most workloads and requires no additional configuration. The default router is especially suited if your workload consists of diverse prompts that don't share significant prefixes, if balancing load across replicas is more important than maximizing cache locality, if you prefer to keep routing logic simple with minimal overhead, or if your prompts don't meaningfully benefit from KV cache reuse.

Use prefix-aware routing when

[Prefix-aware routing](#) optimizes cache locality and can significantly improve throughput, particularly when your workload includes many requests that share long prefixes. This technique is most beneficial when the shared prefix constitutes a significant portion of the total input. Prefix-aware routing is also a strong choice in cases where cache reuse has a direct and meaningful impact on your latency and throughput requirements.



Ask AI

For more information about performance improvements on certain use cases, see the [Anyscale blog post about it](#).

Use custom routing when

[Implement a custom router](#) when you need custom logic for selecting which replica handles each request. Custom routing is appropriate when you want to route based on metrics like GPU memory pressure, batch utilization, or SLOs.

Custom routing provides maximum flexibility but requires more implementation and maintenance effort.

Configure custom autoscaling policies

Beyond routing strategies, you can customize how your deployments scale in response to load. Ray Serve supports custom autoscaling policies at both the deployment level and application level.

Deployment-level policies let you define scaling logic based on metrics such as queue depth, ongoing requests, or custom application metrics. Application-level policies coordinate scaling decisions across multiple deployments simultaneously, which is useful when models share backend resources, have dependencies on each other, or require load-aware coordination.

For detailed information about implementing custom autoscaling policies, see the [Ray Serve custom autoscaling documentation](#).

Monitor routing performance

[Access monitoring tools on Anyscale](#) to evaluate routing performance and engine-level metrics such as time-to-first-token, token throughput, and requests processed per second. For a list of Ray Serve specific metrics, see the [Ray Serve monitoring documentation](#).

Relevant metrics to your router performance include:



- `ray_serve_deployment_queued_queries`: Number of requests waiting for the router to assign them to a replica per deployment

- `ray_serve_num_ongoing_requests_at_replicas`: Number of requests executing on replicas
- `ray_serve_num_router_requests_total`: Total number of requests processed by the router
- `ray_serve_num_scheduling_tasks`: Number of request scheduling tasks in the router

Use these metrics to tune routing parameters and verify that prefix-aware routing is providing the expected performance improvements for your workload.

Summary

In this guide, you learned how to deploy multiple LLMs from a single endpoint and configure request routing to optimize cache locality and improve LLM serving performance. You learned how to deploy multiple models with independent configurations, how the default power-of-two-choices strategy works, how to configure prefix-aware routing on Anyscale services, how to select the right routing policy for your workload, and how to monitor routing performance using Anyscale console metrics.